MediTrack System Design and Implementation Overview

1. Project Summary

MediTrack is a clinic management system written in Core Java. It handles doctors, patients, appointments, billing, persistence, reminders, analytics, and a small recommendation helper.

The project is larger than a basic CRUD exercise, so the code is split into clear layers. Domain objects hold state and core rules, services handle workflows, persistence utilities handle storage, observers handle side effects, and the UI stays thin.

2. Package Layout

The code lives under `com.airtribe.meditrack`.

`entity`

This package contains the domain model:

- `MedicalEntity`
- `Person`
- `Doctor`
- `Patient`
- `Address`
- `Appointment`
- `Bill`
- `BillSummary`
- supporting enums such as `Specialization`, `AppointmentStatus`, and `BillType`

These classes are not just data containers. They also protect their own invariants.

`service`

This package contains the application logic:

- `DoctorService`
- `PatientService`
- `AppointmentService`
- `BillingService`
- `ClinicManagementService`
- `AutoSaveService`

These classes coordinate workflows that span multiple entities.

`service.billing`

This package contains:

- `BillingStrategy`
- `StandardBillingStrategy`
- `InsuranceBillingStrategy`
- `SeniorDiscountBillingStrategy`
- `BillingStrategyFactory`
- `BillFactory`

The billing calculation and the bill-construction step are separate for clarity.

`service.notification`

This package contains:

- `AppointmentObserver`
- `AppointmentEvent`
- `AppointmentEventType`
- `AppointmentSnapshot`
- `ConsoleNotificationObserver`
- `ReminderSchedulerObserver`

These classes keep notifications and reminders out of the scheduling core.

`util`

This package contains:

- `DataStore`
- `PersistenceManager`
- `PersistenceReport`
- `SerializationUtil`
- CSV mappers
- `Validator`
- `DateUtil`
- `IdGenerator`
- `AppConfig`
- `AIHelper`

This is the shared infrastructure layer.

`ui.console` and `ui`

The project has two interfaces:

- the console application, which is the primary path
- a Swing UI, which is included as a secondary interface

Both are intentionally lightweight and call into the service layer for the actual business rules.

3. Domain Model

Shared base classes

`MedicalEntity` provides:

- a stable business ID
- `createdAt`
- `updatedAt`
- shared identity behavior

`Person` adds the shared personal fields used by both `Doctor` and `Patient`.

Doctor

`Doctor` adds:

- specialization
- consultation fee
- slot duration

This data is used directly by scheduling, billing, analytics, and recommendations.

Patient

`Patient` adds:

- insurance flag

- insurance coverage percentage
- address
- allergies

The model also protects nested state through defensive copying and unmodifiable views. `Patient.clone()` also deep copies the address and allergy list.

Appointment

`Appointment` is the most rule-heavy entity in the project. It stores:

- patient ID
- doctor ID
- start time
- duration
- status
- symptoms

The lifecycle is enforced through explicit transitions instead of raw status mutation.

`Appointment.clone()` deep copies the symptom list for the same reason: a copied appointment should not share mutable nested state with the original.

Bill and BillSummary

`Bill` represents the live billing record, including payment state. `BillSummary` is an immutable snapshot used for display and reporting.

This split keeps operational state and presentation output separate.

4. Service Layer

`DoctorService`

Handles:

- doctor creation and update
- deletion
- search by text
- search by specialization
- sorting by fee and name

`PatientService`

Handles:

- patient creation and update
- deletion
- listing and retrieval
- overloaded searches by ID, name, and age

`AppointmentService`

Handles:

- confirmed and pending appointment creation
- confirmation
- cancellation
- completion
- rescheduling
- overlap checks
- doctor availability checks
- slot suggestions
- observer publication

This service is the main scheduling boundary in the project.

`BillingService`

Handles:

- billing eligibility checks
- strategy selection
- bill creation
- bill storage
- bill summary generation

`ClinicManagementService`

Handles cross-entity safety rules, mainly:

- blocking doctor deletion when active appointments still reference that doctor
- blocking patient deletion when active appointments still reference that patient

`AutoSaveService`

Runs periodic saves in the background. It calls the same persistence path used by manual saves.

5. OOP and Design Patterns

Encapsulation

Validation and state rules live inside the model classes. That prevents invalid data from entering the system through alternative paths such as the seeder, loader, or Swing UI.

Abstraction

The main abstractions are:

- `Searchable<T>` for doctor search
- `Payable` for payment behavior
- `AppointmentObserver` for notification side effects
- `CSVUtil<T>` for CSV mapping

`Searchable<T>` provides a shared normalization path for text search. `Payable` keeps the billing contract explicit. `CSVUtil<T>` is a reusable base for CSV mappers so file handling does not get repeated in every entity-specific class.

Inheritance

The inheritance chain is small and direct:

- `MedicalEntity -> Person -> Doctor`
- `MedicalEntity -> Person -> Patient`

Polymorphism

The code uses both forms:

- compile-time polymorphism through overloaded patient search methods
- runtime polymorphism through billing strategies, observers, and CSV mappers

Strategy

Billing uses separate strategies for standard, insured, and senior patients. The pricing rules stay modular.

Factory

`BillingStrategyFactory` chooses the pricing path. `BillFactory` creates the final `Bill`.

Template-style base class

`CSVUtil<T>` works like a template-style base class. It owns the common read and write flow, while the concrete CSV mappers provide the entity-specific header, row serialization, and row parsing logic.

Observer

`AppointmentService` publishes events. The observers react by printing console messages or scheduling reminders.

Facade

`ClinicManagementService` acts as a small facade for cross-entity rules.

Singleton

`IdGenerator` and `AppConfig` are shared single-instance components used across the application. `IdGenerator` manages mutable counters and is initialized lazily. `AppConfig` is lightweight and read-mostly, so it is initialized eagerly.

Snapshot

`BillSummary` and `AppointmentSnapshot` are immutable snapshot types used when a stable view is safer than a live mutable object.

6. Persistence Design

Dual-format storage

The project writes all stores to both CSV and `.ser`. CSV is easy to inspect, while serialization restores full object state quickly.

Centralized persistence

`PersistenceManager` is responsible for save and load operations. The rest of the codebase does not perform file I/O directly.

The actual CSV mappers sit behind `CSVUtil<T>`, which keeps the file-handling algorithm shared and the entity-specific mapping code small.

Fallback reporting

When loading data, the system prefers `.ser`. If deserialization fails, it falls back to CSV and records the result in `PersistenceReport`.

That makes startup behavior visible instead of silent.

The CSV loader preserves trailing empty fields and reports row-level parse failures with file and row information. This is part of the same design goal: persistence errors should be diagnosable instead of vague.

ID reseeding

After loading persisted data, `IdGenerator` is reseeded from existing IDs so new records continue the correct sequence.

7. Concurrency and Background Work

The project uses a small amount of concurrency:

- `AutoSaveService` runs in the background
- `ReminderSchedulerObserver` schedules reminder tasks
- appointment observers are stored in a `CopyOnWriteArrayList`
- ID generation is centralized and thread-safe
- `DataStore<T>` uses synchronized access and snapshot reads

The concurrency model is simple, but it is enough for autosave and reminders without pushing that logic into the UI. Observer callbacks are isolated from each other, and reminder task bookkeeping is synchronized internally.

8. Recommendation Helper

The recommendation feature is deliberately rule-based. `AIHelper` maps symptoms to likely specializations and then asks the appointment logic for real slots.

The feature stays deterministic, testable, and easy to explain.

9. Exception Handling

The project uses a small set of custom runtime exceptions:

- `InvalidDataException` for validation failures and persistence errors
- `AppointmentNotFoundException` for required appointment lookups that fail
- `InputReader.EndOfInputException` as an internal signal for clean console shutdown when input closes

This keeps the application error flow simple. Domain and persistence problems are reported clearly, while the console loop can still exit gracefully in non-interactive runs.

10. UI Approach

The console application is the main interface. The Swing UI is included as a secondary surface.

Both are thin wrappers around the service layer. They collect input, call services, and display results. Business rules stay out of the UI code.

11. Verification

The project includes several verification paths:

- `DemoDataSeeder` creates a fixed baseline dataset
- `TestRunner` checks the main technical behaviors
- `docs/Setup_Instructions.md` records the local build and run steps
- `docs/Demo_Walkthrough.md` documents the main demo flow with screenshots and transcripts
- the UML files document the structure from different angles
- JavaDoc output documents the public API surface

The test runner covers lifecycle rules, persistence fallback, autosave, reminders, safe delete, analytics, billing, and recommendation behavior.

12. Closing Notes

The main goal of the project was not just to make the features work, but to keep the code understandable once those features start interacting. The code therefore leans on clear service boundaries, explicit domain rules, small supporting patterns, and visible verification artifacts.